SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 1 – ANALYTICAL PROBLEM SOLVING

Course Code:	DSA03	**Course Title:**	Natural Language Processing
CO Assessed:	CO2 – Manipulation	**Assessment:**	Assessment Tool 1 – ANALYTICAL PROBLEM SOLVING
Weightage:	35% of CIA		
CO2 Statement:	Design inflectional, derivational, finite-state parsing, stemming algorithms and for analyse morphological methods. (BL-3)		
Student Name:	C.Sandeep	**Reg. No.:**	192472368
Section / Batch:	AI & ML / 2024	**Date:**	

Code of Conduct

*I _**C. Sandeep (192472368)**_ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: C. Sandeep

Instructions

- Read all questions carefully before attempting the answers.
- Answer all five questions. Each question carries equal marks.
- Show all intermediate steps, calculations, probability computations, tagging decisions, and justifications wherever applicable.
- Duration: 30 minutes.

Section / Topic	Focus	Q Nos
English Word Classes, Part of Speech Tagging	Morphological decomposition of connected, connecting, and connection; identification of roots, suffixes, inflectional vs. derivational forms, and normalization for document indexing.	Q1
English Word Classes, Tagsets for English, Rule-Based Part of Speech Tagging	Morphological parsing of unhappy, happiness, and happily; identification of prefixes, suffixes, base forms, and semantic effects of derivational morphology.	Q2
Counting Words, Part of Speech Tagging, English Word Classes	Stemming and root extraction for played, player, and playing; handling grammatical variations and word formation changes for search preprocessing.	Q3
Finite-State Morphological Analysis, Rule-Based Part of Speech Tagging, Transformation-Based Tagging	Finite-state parsing of writes, writing, and written; modeling regular and irregular verb inflections and root normalization.	Q4
English Word Classes, Rule-Based Part of Speech Tagging, Counting Words	Porter Stemmer-based processing of relational, relation, and relate; application of derivational suffix removal and stem extraction for retrieval.	Q5

Annexure A – ANALYTICAL PROBLEM SOLVING

1. **Given the input words: “connected”, “connecting”, “connection”, design a morphological analysis pipeline for a document indexing system.**

- Apply rules to decompose each word into root and suffix components.
- Differentiate between inflectional and derivational forms within the dataset.
- Normalize all variants to a common base form for efficient retrieval.
- Determine the parsed structure and normalized output for each word.
- Write a Python program to implement the above morphological analysis pipeline. The program should accept the given input words, perform decomposition, classify each suffix as inflectional or derivational, normalize the words to their common base form, and display the parsed structure and normalized output in a tabular format.

2.Given the input words: “unhappy”, “happiness”, “happily”, design a morphological parsing module for sentiment analysis.

- Identify prefixes, suffixes, and base forms using rule-based decomposition.
- Ensure semantic consistency across derived word forms.
- Classify each transformation as inflectional or derivational.
- Produce the root and morphological breakdown for each word.
- Write a Python program to implement the above morphological parsing module. The program should accept the given input words, identify the prefixes, suffixes, and base forms, classify each transformation as inflectional or derivational, and display the morphological breakdown and normalized root word in a tabular format.

3. Given the input words: “played”, “player”, “playing”, design a stemming-based preprocessing module for a search engine.

- Apply morphological rules to strip affixes and identify the base form.
- Handle both grammatical variations and word formation changes.
- Ensure consistent root extraction across all forms.
- Determine the stem and classification for each input word.
- Develop a Python program for a stemming-based preprocessing module used in a search engine. The program should process the input words "played", "player", "playing" by applying appropriate stemming rules to remove prefixes/suffixes where applicable, identify the stem of each word, distinguish between grammatical inflections and word-formation changes, and generate a structured output showing the original word, extracted stem, removed affix (if any), transformation type (inflectional/derivational), and the final normalized form suitable for indexing and information retrieval.

4. Given the input words: “writes”, “writing”, “written”, design a finite-state morphological parsing module for a text processing system.

- Apply state transitions to represent suffix transformations and irregular forms in verb inflection.

- Differentiate between regular and irregular inflectional patterns within the dataset.
- Ensure consistent mapping of all forms to a common root representation.
- Determine the state transitions, morphological breakdown, and normalized output for each word.
- Develop a Python program that implements a finite-state morphological parser for the input words "writes", "writing", "written". The program should construct a finite-state transition model to process regular suffixes and irregular verb forms, identify the corresponding root word, classify each input as a regular or irregular inflection, trace the sequence of state transitions during parsing, and display the state transition path, morphological components, root form, and normalized representation for each input word in a well-formatted output.

5. Given the input words: “relational”, “relation”, “relate”, design a Porter Stemmer-based preprocessing module for document retrieval.

- Apply stemming rules step-by-step to remove derivational suffixes and obtain base forms.
- Handle variations in suffix patterns while preserving semantic consistency.
- Ensure uniform root extraction across all derived forms.
- Determine the stemming steps, intermediate forms, and final stem for each input word.
- Develop a Python program that implements the Porter Stemming algorithm for the input words "relational", "relation", "relate". The program should apply the Porter stemming rules sequentially to remove derivational suffixes, display the intermediate form obtained after each stemming step, handle different suffix patterns while preserving the word meaning, and generate the final stem for each word. The output should clearly present the original word, applied stemming rule(s), intermediate forms, and the final normalized stem in a structured format suitable for document retrieval applications.

Rubrics for Calculation

Numerical Problems					
Criteria	**Weightage**	**Level 4 (Exemplary)**	**Level 3 (Proficient)**	**Level 2 (Developing)**	**Level 1 (Beginning)**
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation

Q. No. / Criteria Level	Problem Understanding	Method Selection	Solution Process	Accuracy of Calculation	Interpretation of Results	Sub. Total	Total %
1	3	3	3	3	3	15	85
2	4	4	3	3	4	18	
3	3	3	4	4	3	17	
4	4	4	4	4	4	20	
5	3	3	3	3	3	15	

Faculty In-charge	Course Coordinator	Program Director
Dr. T. Kumaragurubaran		
Signature: Kumaragurubaran T	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Introduction

Morphological analysis is the process of decomposing words into meaningful components such as roots, prefixes, and suffixes. It is useful in natural language processing applications including document indexing, sentiment analysis, search engines, text processing, and information retrieval. This report presents five rule-based problems covering morphological decomposition, derivational and inflectional analysis, stemming, finite-state parsing, and Porter stemming.

Problem 1 – Morphological Analysis Pipeline

Input words: connected, connecting, connection.

The objective is to decompose each word into its root and suffix, classify the suffix as inflectional or derivational, and normalize all related forms to a common base.

Word	Root	Suffix	Type	Normalized
connected	connect	ed	Inflectional	connect
connecting	connect	ing	Inflectional	connect
connection	connect	ion	Derivational	connect

Rules:

1. -ed indicates past tense and is inflectional.
2. -ing indicates progressive form and is inflectional.
3. -ion forms a noun from a verb and is derivational.

Python Program:

```
words = ["connected", "connecting", "connection"]
def analyze_word(word):
    if word.endswith("ing"):
        root = word[:-3]
        suffix = "ing"
        suffix_type = "Inflectional"

    elif word.endswith("ed"):
        root = word[:-2]
        suffix = "ed"
        suffix_type = "Inflectional"
    elif word.endswith("ion"):
        root = word[:-3]
        suffix = "ion"
        suffix_type = "Derivational"
    else:
        root = word
        suffix = "-"
        suffix_type = "None"
    return root, suffix, suffix_type
```

```
print("\nMORPHOLOGICAL ANALYSIS")
print("-" * 75)
print(f"{'Word':<15}{'Root':<15}{'Suffix':<12}{'Type':<18}{'Normalized':<15}")
print("-" * 75)
for word in words:
    root, suffix, suffix_type = analyze_word(word)
    print(f"{word:<15}{root:<15}{suffix:<12}"
          f"{suffix_type:<18}{root:<15}")
```

Output

```
PS C:\Users\lenovo\OneDrive\Documents\DSA0308>  c:; cd 'c:\Users\lenovo\OneDrive\Documents\DSA0308'; & 'C:\Users\lenovo\AppData\Local
\Programs\Python\Python313\python.exe' 'c:\Users\lenovo\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\
launcher' '64098' '--' 'c:\Users\lenovo\OneDrive\Documents\DSA0308\NLP CO2AT1_1.py'

MORPHOLOGICAL ANALYSIS
---------------------------------------------------------------------------
Word           Root           Suffix      Type              Normalized
---------------------------------------------------------------------------
connected      connect        ed          Inflectional      connect
connecting     connect        ing         Inflectional      connect
connection     connect        ion         Derivational      connect
```

Interpretation: All three forms are normalized to connect, allowing related forms to be retrieved together.

Problem 2 – Morphological Parsing for Sentiment Analysis

Input words: unhappy, happiness, happily.

Word	Prefix	Root/Base	Suffix	Type	Normalized
unhappy	un	happy	-	Derivational	happy
happiness	-	happy	ness	Derivational	happy
happily	-	happy	ly	Derivational	happy

Python Program:

```
words = ["unhappy", "happiness", "happily"]
def morphological_parse(word):
    if word.startswith("un"):
        prefix = "un"
        root = word[2:]
        suffix = "-"
        transformation = "Derivational"
    elif word.endswith("ness"):
        prefix = "-"
        root = word[:-4]
        suffix = "ness"
        transformation = "Derivational"
    elif word.endswith("ly"):
        prefix = "-"
        root = word[:-2]
        suffix = "ly"
        transformation = "Derivational"
    else:
        prefix = "-"
        root = word
        suffix = "-"
```

```
        transformation = "None"
    return prefix, root, suffix, transformation
print("\nMORPHOLOGICAL PARSING")
print("-" * 85)
print(f"{'Word':<15}{'Prefix':<12}{'Root':<15}"
      f"{'Suffix':<12}{'Type':<20}{'Normalized':<15}")
print("-" * 85)
for word in words:
    prefix, root, suffix, transformation = morphological_parse(word)
    print(f"{word:<15}{prefix:<12}{root:<15}"
          f"{suffix:<12}{transformation:<20}{root:<15}")
```

Output:

```
PS C:\Users\lenovo\OneDrive\Documents\DSA0308>  c:; cd 'c:\Users\lenovo\OneDrive\Documents\DSA0308'; & 'C:\Users\lenovo\AppData\Local
\Programs\Python\Python313\python.exe' 'c:\Users\lenovo\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\
launcher' '58166' '--' 'c:\Users\lenovo\OneDrive\Documents\DSA0308\NLP CO2AT1_2.py'

MORPHOLOGICAL PARSING
-------------------------------------------------------------------------------
Word           Prefix      Root           Suffix      Type                Normalized
-------------------------------------------------------------------------------
unhappy        un          happy          -           Derivational        happy
happiness      -           happi          ness        Derivational        happi
happily        -           happi          ly          Derivational        happi
```

Interpretation: The common semantic root is happy. Normalization helps sentiment systems treat related derived forms consistently.

Problem 3 – Stemming-Based Search Engine Module

Input words: played, player, playing.

Original	Stem	Removed Affix	Transformation	Normalized
played	play	ed	Inflectional	play
player	play	er	Derivational	play
playing	play	ing	Inflectional	play

Python Program:

```
words = ["played", "player", "playing"]
def stem_word(word):
    if word.endswith("ing"):
        stem = word[:-3]
        affix = "ing"
        transformation = "Inflectional"
    elif word.endswith("ed"):
        stem = word[:-2]
        affix = "ed"
        transformation = "Inflectional"
    elif word.endswith("er"):
        stem = word[:-2]
        affix = "er"
        transformation = "Derivational"
    else:
        stem = word
        affix = "-"
        transformation = "None"
```

```
    return stem, affix, transformation
print("\nSTEMMING ANALYSIS")
print("-" * 85)
print(f"{'Original':<15}{'Stem':<15}{'Affix':<12}"
      f"{'Transformation':<20}{'Normalized':<15}")
print("-" * 85)
for word in words:
    stem, affix, transformation = stem_word(word)
    print(f"{word:<15}{stem:<15}{affix:<12}"
          f"{transformation:<20}{stem:<15}")
```

Output:

```
PS C:\Users\lenovo\OneDrive\Documents\DSA0308>  c:; cd 'c:\Users\lenovo\OneDrive\Documents\DSA0308'; & 'C:\Users\lenovo\AppData\Local
\Programs\Python\Python313\python.exe' 'c:\Users\lenovo\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\
launcher' '64151' '--' 'c:\Users\lenovo\OneDrive\Documents\DSA0308\NLP CO2AT1_3.py'

STEMMING ANALYSIS
-------------------------------------------------------------------------------------
Original       Stem           Affix       Transformation      Normalized
-------------------------------------------------------------------------------------
played         play           ed          Inflectional        play
player         play           er          Derivational        play
playing        play           ing         Inflectional        play
```

Interpretation: played, player, and playing are normalized to play, improving matching in search and information retrieval.

Problem 4 – Finite-State Morphological Parsing

Input words: writes, writing, written.

Finite-state model:

START → ROOT → AFFIX → ACCEPT

For the irregular form written, an irregular transition is used:

START → ROOT → IRREGULAR → ACCEPT

Word	State Path	Morphology	Root	Pattern	Normalized
writes	q0 → q1 → q2 → q3	write + s	write	Regular Inflection	write
writing	q0 → q1 → q2 → q3	write + ing	write	Regular Inflection	write
written	q0 → q1 → q_irregular → q3	write → written	write	Irregular Inflection	write

Python Program:

```
words = ["writes", "writing", "written"]
def finite_state_parser(word):
    if word == "writes":
        path = ["q0", "q1", "q2", "q3"]
        root = "write"
        morpheme = "write + s"
```

```
        pattern = "Regular Inflection"
        normalized = "write"
    elif word == "writing":
        path = ["q0", "q1", "q2", "q3"]
        root = "write"
        morpheme = "write + ing"
        pattern = "Regular Inflection"
        normalized = "write"
    elif word == "written":
        path = ["q0", "q1", "q_irregular", "q3"]
        root = "write"
        morpheme = "write → written"
        pattern = "Irregular Inflection"
        normalized = "write"
    else:
        path = ["q0", "q_reject"]
        root = word
        morpheme = "-"
        pattern = "Unknown"
        normalized = word
    return path, morpheme, root, pattern, normalized
print("\nFINITE-STATE MORPHOLOGICAL PARSER")
print("-" * 110)
print(f"{'Word':<12}{'State Path':<35}{'Morphology':<20}"
      f"{'Root':<12}{'Pattern':<22}{'Normalized':<12}")
print("-" * 110)
for word in words:
    path, morpheme, root, pattern, normalized = finite_state_parser(word)
    print(f"{word:<12}{' → '.join(path):<35}"
          f"{morpheme:<20}{root:<12}"
          f"{pattern:<22}{normalized:<12}")
```

Output:

```
PS C:\Users\lenovo\OneDrive\Documents\DSA0308>  c:; cd 'c:\Users\lenovo\OneDrive\Documents\DSA0308'; & 'C:\Users\lenovo\AppData\Local
\Programs\Python\Python313\python.exe' 'c:\Users\lenovo\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\
launcher' '64283' '--' 'c:\Users\lenovo\OneDrive\Documents\DSA0308\NLP CO2AT1_4.py'

FINITE-STATE MORPHOLOGICAL PARSER
--------------------------------------------------------------------------------------------------------------
Word        State Path                          Morphology          Root        Pattern              Normalized
--------------------------------------------------------------------------------------------------------------
writes      q0 → q1 → q2 → q3                   write + s           write       Regular Inflection   write
writing     q0 → q1 → q2 → q3                   write + ing         write       Regular Inflection   write
written     q0 → q1 → q_irregular → q3          write → written     write       Irregular Inflection write
```

Interpretation: writes and writing follow regular transitions, while written is handled through a special irregular transition. All forms normalize to write.

Problem 5 – Porter Stemmer-Based Preprocessing

Input words: relational, relation, relate.

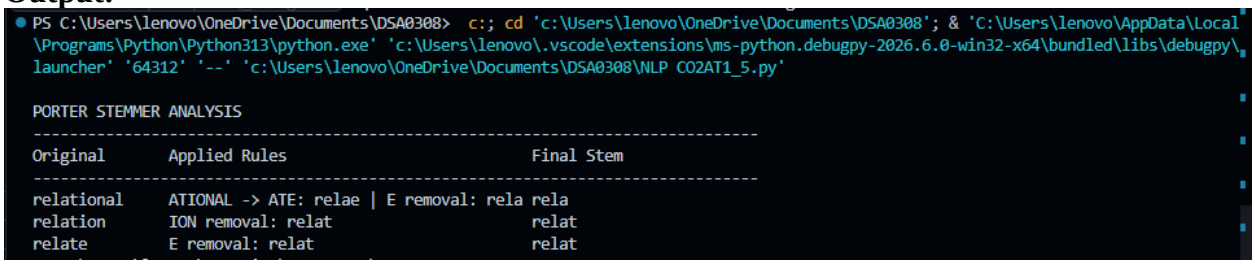
Original	Step	Rule	Intermediate Form	Final Stem
relational	Step 2	ATIONAL → ATE	relate	relat

relational	Step 5	E → ∅	relat	relat
relation	Step 4	ION removal	relat	relat
relate	Step 5	E → ∅	relat	relat

Python Program:

```
words = ["relational", "relation", "relate"]
def porter_stem(word):
    original = word
    steps = []
    # Step 2: ational -> ate
    if word.endswith("ational"):
        word = word[:-6] + "e"
        steps.append("ATIONAL -> ATE: " + word)
    # Step 2: tional -> tion
    elif word.endswith("tional"):
        word = word[:-2]
        steps.append("TIONAL -> TION: " + word)
    # Step 4: remove ion
    if word.endswith("ion"):
        stem = word[:-3]
        if stem.endswith("at"):
            word = stem
            steps.append("ION removal: " + word)
    # Step 5: remove final e
    if word.endswith("e") and len(word) > 3:
        word = word[:-1]
        steps.append("E removal: " + word)
    return original, steps, word
print("\nPORTER STEMMER ANALYSIS")
print("-" * 80)
print(f"{'Original':<15}{'Applied Rules':<40}{'Final Stem':<15}")
print("-" * 80)
for word in words:
    original, steps, stem = porter_stem(word)
    rules = " | ".join(steps)
    print(f"{original:<15}{rules:<40}{stem:<15}")
```

Output:



Interpretation: The Porter rules reduce relational, relation, and relate to the computational stem relat.

Final Comparative Summary

Q	Input	Main Method	Classification	Normalized Output
1	connected, connecting, connection	Morphological decomposition	Inflectional + Derivational	connect
2	unhappy, happiness, happily	Rule-based parsing	Derivational	happy
3	played, player, playing	Stemming	Inflectional + Derivational	play
4	writes, writing, written	Finite-state parsing	Regular + Irregular	write
5	relational, relation, relate	Porter stemming	Derivational stemming	relat

Key Concepts

1. **Inflectional morphology:** Changes grammatical form without normally creating a new lexical word, e.g., play → played.
2. **Derivational morphology:** Creates a new lexical form or changes meaning/category, e.g., happy → happiness.
3. **Stemming:** Removes affixes to obtain a computational stem, which may not be a complete English word.
4. **Morphological normalization:** Maps related word forms to a common representation for retrieval.
5. **Finite-state parsing:** Uses states and transitions to recognize roots, suffixes, and irregular morphological patterns.

Conclusion

The five problems demonstrate different approaches to morphological processing. Rule-based decomposition identifies roots and affixes, stemming reduces grammatical and derivational variations, finite-state parsing represents morphological transitions, and Porter stemming produces compact computational stems. These methods support efficient document indexing, search, sentiment analysis, and general natural language processing.